A Simple Batch ETL Pipeline

1. What is a Simple Batch ETL Pipeline?

A **simple batch ETL pipeline** runs scheduled jobs periodically using:

```
None
Crontab Scheduler

SQL Tables

Scripts (Python / Shell / Java etc.)
```

Used for small internal workloads.

2. Core Components

```
None
Cron          → Triggers jobs on schedule

cron_dag      → Stores dependency graph

cron_jobs     → Stores job metadata

Scripts       → Perform ETL tasks
```

3. Example SQL Table: DAG Structure

```SQL
CREATE TABLE cron_dag (
  id INT,
  parent_id INT,
  PRIMARY KEY (id),
  FOREIGN KEY (parent_id) REFERENCES cron_dag(id)
);
```

Used to define dependencies.

Example:

```None
Job 2 depends on Job 1

Job 3 depends on Job 2
```

4. Example SQL Table: Job Status

```SQL
CREATE TABLE cron_jobs (
  id INT,
  name VARCHAR(255),
  updated_at INT,
  PRIMARY KEY (id)
);
```

Stores:

```None
Job Name
Last successful run time
Status tracking
```

5. Example Cron Schedule

```None
0 * * * * ~/cron_dag/dag/first_node.py
0 * * * * ~/cron_dag/dag/second_node.py
```

Means:

```
None
Run every hour at minute 0
```

6. How Each Script Works

Every script follows:

```
None
1. Check parent jobs completed

2. Trigger monitoring if needed

3. Execute ETL logic

4. Update status table
```

7. Example Flow

```
None
first_node.py  → Extract sales data

second_node.py → Clean and load reports
```

Where second job starts only after first succeeds.

8. Simple Architecture

```
Cron
 ↓
Python Scripts
 ↓
SQL Metadata Tables
 ↓
Destination DB / Files
```

9. Advantages

```
Very easy to build

Low cost

Good for small teams

Uses existing tools

Simple debugging
```

10. Main Problems

A. Single Host Failure

```
None
```

All jobs run on one machine

If machine crashes → pipeline stops

B. Limited Compute Power

```
None
```

Many jobs at same time

CPU / RAM may become insufficient

C. Limited Storage

```
None
```

Logs / temp files / exports may fill disk

D. No Horizontal Scaling

```
None
Cannot easily distribute jobs across servers
```

E. Retry Problems

Example:

```
None
Send 1 million notifications

Job fails after 800,000

Retry sends duplicates
```

Need **idempotent jobs**.

F. No Validation

```
None
Python script uses Job ID=7

Database stores Job ID=8

Dependency breaks
```

G. No GUI

```
No dashboard

No DAG visualization

No manual trigger UI
```

H. Missing Monitoring

Need alerts for:

```
Job failed

Job delayed

Server crashed

Long runtime

Dependency blocked
```

11. Better Production Alternatives

Use dedicated schedulers:

```
Apache Airflow

Luigi

Prefect

Dagster
```

They provide:

```
Web UI

Retries

Parallelism

Logs

Alerts

Distributed workers

DAG management
```

12. How to Horizontally Scale

Instead of one host:

```
None
Scheduler Node

   ↓

Worker 1

Worker 2

Worker 3
```

Use queue-based execution.

13. Modern HLD Design

```
None
Airflow Scheduler

      ↓

Task Queue

      ↓

Worker Cluster

      ↓

Warehouse / DB / Files
```

14. When This Simple Design is Fine

Use simple cron ETL when:

```
None
Small startup

Internal reports

Low criticality jobs

Few daily tasks

Single engineer setup
```

15. When to Avoid

Avoid when:

```
None
Many jobs

Need retries

Need HA

Need SLA guarantees

Need real-time visibility

Large team operations
```

16. Interview Talking Points

If asked basic ETL scheduler:

```
None
Use cron + scripts + SQL metadata initially

Then migrate to Airflow for scale

Need retries, monitoring, idempotency

Separate scheduler and workers
```

17. One-Line Summary

A simple batch ETL pipeline using cron, SQL tables, and scripts is low-cost and easy to build, but suffers from poor scalability, single-point failure risk, weak monitoring, and limited production readiness.